



Peer-to-Peer Directory Synchronization with Lightweight Version Control

Arihant Jain

Su Huizhe

Nizar Soufiya

Shengkang Duan

EPITA Graduate School of Computer Science

Advisor

Thomas Broussard

June 22, 2026

Declaration of Own Work

This work has been composed solely by us.

This work has not been accepted in any previous application for a degree or diploma, by Arihant Jain, Su Huizhe, Nizar Soufiya, and Shengkang Duan, or anyone else.

The work of which this is a record has been wholly done by us.

All extracts, including those created by generative AI, have been distinguished by quotation marks and the sources of our information have been specifically acknowledged and documented with in-text citations and in the bibliography.

All prompts with responses for any use Generative AI must be included in a separate MS Word document, clearly referencing its appearance in the bibliography and allowing the reader to easily associate the extract you used with the prompt. Here is how APA cites Generative AI:

<https://apastyle.apa.org/blog/how-to-cite-chatgpt>

Date: _____

Name: _____

Signature: _____

Name: _____

Signature: _____

Name: _____

Signature: _____

Name: _____

Signature: _____

Acknowledgements

We would like to thank our advisor for their guidance throughout this project. We also thank the EPITA faculty and Action Learning programme coordinators for providing the framework and resources that made this project possible. Finally, we thank each other for the late debugging sessions and the willingness to learn new languages and tools on a tight timeline.

Abstract

Small collaborative teams, particularly university students working on temporary group projects, face a recurring problem: sharing and tracking project files without dedicated infrastructure. Cloud storage services require external servers and internet connectivity. Version control systems demand technical expertise and are designed for source code. Peer-to-peer file synchronization tools lack version history, meaning a single accidental overwrite can propagate to every connected device with no way to undo it.

This thesis proposes a peer-to-peer directory synchronization system with built-in version safety. The system discovers peers automatically on a local network using mDNS/DNS-SD, synchronizes file changes over direct TCP connections, and creates local backups before every synchronized overwrite. Conflict resolution uses a Last-Write-Wins policy paired with Lamport clock timestamps, a combination that keeps the implementation simple while ensuring no data is permanently lost.

The system is implemented as two cooperating daemons: a Go networking component handling peer discovery and synchronization coordination, and a C++ storage component handling file watching, content hashing, and disk I/O. A Java graphical frontend provides user visibility into synchronization status and version history.

Contents

Declaration of Own Work	1
Acknowledgements	2
Abstract	3
1 Introduction	7
1.1 Motivations	7
1.2 Context of the Project	8
1.3 Objectives	8
1.4 Overview of the Thesis	9
2 Literature Review	10
2.1 The Collaborative File Management Landscape	10
2.2 Synchronization in Distributed Systems	11
2.2.1 Ordering Events Without a Shared Clock	12
2.2.2 The CAP Theorem and Why It Matters	12
2.2.3 Resolving Conflicts	13
2.2.4 Propagating Updates Efficiently	14
2.3 Identified Gap	14
2.4 Positioning the Project	15
2.5 Summary	16
3 State of the Art and Design Choices	17
3.1 Summary	17
3.2 Zero-Configuration Peer Discovery	17
3.3 Event Ordering with Lamport Clocks	18
3.4 Availability Over Consistency	19

3.5	Last-Write-Wins with Version Backups	19
3.6	File Change Detection	20
3.7	Data Integrity and Network Transport	20
4	System Architecture	22
4.1	System Architecture Anticipated	22
4.1.1	Component Overview	22
4.1.2	Architecture Diagram	23
4.1.3	Synchronization Data Flow	23
4.1.4	Metadata Storage	24
4.1.5	Communication Protocols	24
4.2	System Architecture Updated	25
5	Methodology	26
5.1	Methodology Anticipated	26
5.1.1	Team Structure and Roles	26
5.1.2	Development Process	27
5.1.3	Tools and Instruments	28
5.1.4	Research Design	28
5.1.5	Action Plan	29
5.2	Methodology Updated	30
6	Results	31
6.1	Development Anticipated	31
6.2	Development Updated	32
7	Conclusions	33
	References	35
	Appendices	38
A	Detailed Comparison Tables	39

B Flesch Readability Statistics	42
List of Figures	43
List of Tables	44

Chapter 1

Introduction

1.1 Motivations

The idea for this project came directly from discussions within our team. Arihant wanted to work on distributed systems, but the specific project proposal and application concept were developed by Su and Nizar based on real problems they observed.

Specifically, Nizar observed this difficulty during EPITA’s *Challenge Week*. In a high-pressure team setting, group members who were unfamiliar with Git spent a lot of time troubleshooting setup issues, broken repositories, and merge conflicts. These technical hurdles regularly delayed collaboration, proving that for non-programmers or short-term work, Git is too complex to be practical.

At the same time, we saw the limits of simpler sharing methods. During a database course last semester, three of our team members ended up with four different versions of an ER diagram file. Reconciling those changes manually across email, WhatsApp, and USB drives was error-prone, and the final submission used an outdated version.

Existing tools address parts of this problem. Google Drive (Google, 2026) and Dropbox (Dropbox, 2026) synchronize files through cloud servers but require internet access, accounts, and have storage quotas. Git (Chacon & Straub, 2014) tracks changes perfectly but requires advanced training in branches and staging. Syncthing (Syncthing Community, 2026) handles peer-to-peer transfers serverlessly, but it treats file version history as an afterthought and lacks a simple interface to undo accidental overwrites.

Our goal was to build a tool that does all three: synchronize files automatically on a local network, protect files with automatic version backups before overwrites, and run with zero configuration or technical overhead.

1.2 Context of the Project

This project was developed as part of the Action Learning programme at EPITA, a programme that asks students to identify a real problem and build a solution in a team of four. Our team has two backend developers working in Go and C++, and two frontend and CI/CD developers working in Java with GitHub Actions.

The idea came from our own frustration. Rather than building something theoretical, we decided to solve the file-sharing problem we kept running into. The system we set out to build should feel invisible: put files into a folder on your laptop, and they show up on your teammates' laptops. Change a file, and everyone gets the update. If an update accidentally overwrites something, a backup of the old version is waiting in a local folder.

The Action Learning context shaped how we worked. With a fixed timeline and a small team, every design decision had to balance ambition against what could actually be built and tested in the available time.

1.3 Objectives

The project has three objectives:

1. **Build a serverless synchronization system.** Peers discover each other automatically on a local network using mDNS/DNS-SD (the same protocols that let printers appear on your Wi-Fi without configuration). Files synchronize over direct TCP connections. No server, no accounts, no internet required.
2. **Protect against accidental data loss.** Before any synchronized overwrite, the system saves the existing file into a local backup directory. Conflict resolution uses a Last-Write-Wins rule based on Lamport clock timestamps: predictable, automatic, and reversible.
3. **Test two specific claims:**

H_1 : Average synchronization latency under one second on a standard WLAN for files up to 10 MB, with no cloud infrastructure.

H_2 : For 3 to 5 peers, Lamport clocks with LWW are sufficient to maintain consistency and support rollback, without needing vector clocks or CRDTs.

1.4 Overview of the Thesis

Chapter 2 reviews the academic literature on distributed synchronization and collaborative file management. It discusses the trade-offs between different approaches and identifies the specific gap our project addresses.

Chapter 3 covers the state of the art: the research findings and technical decisions that determined our architecture, from protocol selection to conflict resolution strategy.

Chapter 4 describes the system architecture, how the Go and C++ components divide responsibilities, and how data flows through the system.

Chapter 5 explains the methodology, including our team structure, development process, tools, and the action plan for Action Learning Week.

Chapter 6 lists the expected development outcomes. After Action Learning Week, it will also cover what actually happened.

Chapter 7 presents the conclusions, summarises the contributions, and suggests directions for future work.

The appendices contain detailed comparison tables that support the analysis in Chapter 2.

Chapter 2

Literature Review

This chapter covers the research that shaped our project. The goal is not to list tools or summarize papers, but to explain what the existing literature taught us and where it left questions that our project tries to answer.

The research was divided among the team. Arihant and Huizhe took on distributed systems theory: consistency models, replication strategies, and synchronization algorithms. They focused on the academic side, reading papers by Lamport, Shapiro, and Saito. Nizar and Shengkang surveyed existing tools and local networking technologies, testing products like Syncthing and comparing cloud platforms. Each week, we met to present what we found and argue about what mattered for our design. Some of the best design decisions came out of those arguments.

2.1 The Collaborative File Management Landscape

Collaborative file management tools can be grouped into four categories. None of them does everything well, and understanding where each one falls short is what pointed us toward our project.

Centralized cloud platforms (Google Drive (Google, 2026), Dropbox (Dropbox, 2026), Nextcloud) are the tools most students reach for first. They synchronize files through a central server, handle multiple devices, and have friendly interfaces. The problems are structural, not cosmetic. Everything depends on the server. If the internet is down, or if the campus Wi-Fi drops, you cannot sync. If the provider changes their pricing or storage limits, you adjust or leave. Dropbox’s own engineering team rebuilt their sync engine from scratch to handle the complexity of tracking file states across devices (Jayakar, 2020), which illustrates how

difficult reliable synchronization is even with centralised infrastructure. Version history exists, but it is time-limited (30 days on Google Drive, for example) and not designed for tracking the evolution of a project.

Version control systems (Git (Chacon & Straub, 2014), SVN, Perforce) solve the history problem completely. Git stores every version of every file, supports branching, and has detailed merge conflict resolution. But Git was built for software developers. The staging-committing-pushing workflow is not intuitive for someone who just wants to share a presentation or a dataset. In our own team, two out of four members had never used Git before this project, which is exactly the audience we are designing for.

Peer-to-peer synchronization tools (Syncthing (Syncthing Community, 2026), Resilio Sync (Resilio, Inc., 2026)) take a different approach. They synchronize files directly between devices, no server needed. Syncthing in particular is open source, works on most platforms, and handles any file type. But when Nizar tested it for a week, he noticed a gap: if he accidentally saved a file with errors, it immediately replaced the file on all connected devices. Syncthing does have a versioning feature, but it is not on by default and the interface for recovering old versions is buried in the settings. Versioning is an afterthought, not part of the core design.

Local sharing (AirDrop, Nearby Share) is the simplest category. Select a file, tap a device, done. But these are one-shot transfers. There is no persistent connection, no synchronization, and no history. They solve a different problem.

The pattern is clear: each category does well on some dimensions and poorly on others. No single tool provides automatic synchronization, version safety, and zero configuration at the same time. The detailed comparison tables in Appendix A show this across five evaluation dimensions.

2.2 Synchronization in Distributed Systems

Rather than providing a comprehensive survey of distributed systems literature, this section highlights the specific synchronization challenges and design trade-offs that guided our implementation.

2.2.1 Ordering Events Without a Shared Clock

The first problem any distributed system faces is ordering. When things happen on different machines, how do you decide what came first? Physical clocks are unreliable because machines drift.

Lamport (1978) proposed a simple solution: logical clocks. Each process keeps a counter and increments it on every event. When sending a message, the sender attaches its counter. The receiver updates its own counter to be at least as high. This creates a *partial ordering*: if event *A* causally led to event *B*, then *A*'s timestamp will always be lower.

The word “partial” matters. Lamport clocks cannot tell you whether two events are concurrent, that is, whether they happened independently with no causal connection. Fidge (1988) and Mattern independently proposed vector clocks to detect concurrency. Each node keeps an array of counters, one per participant. Vector clocks can identify concurrent events, but the counter array grows with the number of peers. Earlier, Parker et al. (1983) had introduced version vectors for detecting mutual inconsistency between replicas, the formal basis for recognising when two copies of the same data have diverged.

For our system, this trade-off was the first real design decision. With 3 to 5 peers on a local network, vector clocks would mean carrying a small array of integers. The overhead is tiny. But vector clocks also require knowing all participants, and adding or removing a peer requires updating every node's vector. Since our system is designed for temporary groups where members join and leave, Lamport clocks are simpler to manage. The cost is that we cannot detect true concurrency. If two peers edit the same file at the exact same moment, one version will silently overwrite the other. The backup mechanism makes this acceptable, but it is a real limitation.

2.2.2 The CAP Theorem and Why It Matters

Gilbert and Lynch (2002) proved that a distributed system cannot simultaneously guarantee consistency, availability, and partition tolerance. During a network partition (which on a campus Wi-Fi happens every time a laptop lid closes), you must choose: either block operations

until the partition heals (consistency), or allow operations to continue and reconcile later (availability).

In our case, we have to prioritize availability. Student laptops disconnect constantly: Wi-Fi drops, people move between classrooms, or a battery dies. If the sync tool blocked edits whenever a teammate was offline, it would be unusable. We therefore accept eventual consistency, allowing each peer to edit files offline and sync changes once they reconnect.

Vogels (2009) argued that eventual consistency is the right model for applications where reading slightly stale data is harmless. A teammate seeing a file that is ten seconds out of date causes no damage. Losing a file because the system refused to save it during a disconnection does.

2.2.3 Resolving Conflicts

When two peers edit the same file while disconnected, the system must decide what happens when they reconnect. The literature offers several approaches, each with trade-offs.

Operational transformation (OT) (Sun & Ellis, 1998) tracks individual editing operations (insert character at position 5, delete characters 3 through 7) and transforms them so they can be applied in any order. Google Docs relies on this. OT is powerful for text, but it requires operation-level logging. For binary files like images, spreadsheets, or compiled code, there are no “operations” to transform.

CRDTs (Conflict-free Replicated Data Types) (Shapiro, Preguiça, Baquero, & Zawirski, 2011) are data structures that mathematically guarantee convergence. Shapiro et al. proved that any data type whose merge operation is commutative, associative, and idempotent will converge regardless of operation order. This is a strong result. But applying CRDTs to file metadata requires defining what “merge” means for each field in the metadata schema, and testing the merge logic thoroughly.

Last-Write-Wins (LWW) is the simplest policy: keep the version with the highest timestamp. One version is discarded. The Bayou project (Petersen, Spreitzer, Terry, Theimer, & Demers, 1995) demonstrated that LWW becomes safe when combined with backup storage: before overwriting a file, save the old copy somewhere. Terry et al. (1994) added “session

guarantees,” ensuring that a user always sees the results of their own writes, even if other peers’ updates have not arrived.

After reading these papers, the team debated for two meetings before settling on LWW with backups. CRDTs would be technically better, but implementing and testing them correctly within our timeline was unrealistic. Saito and Shapiro’s (2005) survey of optimistic replication confirmed that our approach, optimistic local updates with deferred reconciliation and rollback, is a well-documented pattern for weakly connected systems. That reassured us. The metadata schema we designed can support CRDTs in a future version, using register-based merge semantics from Shapiro et al. (2011), so the option remains open.

2.2.4 Propagating Updates Efficiently

Tridgell and Mackerras (1996) showed that rolling checksums allow two machines to identify which *blocks* of a file have changed and transfer only the differences. This is the core idea behind rsync, and it reduces bandwidth use dramatically for large files with small changes.

Demers et al. (1987) studied a separate problem: how to reliably spread updates across a group of peers without a central coordinator. Their “epidemic” (gossip) algorithms model updates as a contagion that spreads from peer to peer until everyone has it. The approach is probabilistic but robust, and it scales well.

For our initial version, we transfer whole files. It is simpler to implement, simpler to debug, and for files under a few megabytes on a local network, the performance penalty is small. The architecture is designed to support both optimizations later: block-level transfers in the C++ daemon, and gossip propagation in the Go coordinator.

2.3 Identified Gap

Comparing existing tools across five dimensions (membership management, synchronization capability, history management, file type support, and filesystem transparency) shows a consistent pattern. The full tables are in Appendix A; here is the summary:

- Cloud platforms score well on synchronization and usability, but poorly on history man-

agement and offline use.

- Version control systems score well on history, but poorly on synchronization automation and accessibility for non-technical users.
- P2P sync tools score well on synchronization and usability, but poorly on history management.
- Local sharing tools score well on usability but offer no synchronization or history at all.

The gap is not that any single feature is missing from the market. It is that no tool scores well across all five dimensions simultaneously. Each existing tool was optimised for a different audience (enterprise workers, developers, power users, casual sharers), leaving a gap for small, temporary teams who want something that “just works” on a local network with version safety included.

2.4 Positioning the Project

Our system targets this gap. It differs from existing tools in four concrete ways:

- **LAN-only, serverless operation.** Peer discovery uses mDNS/DNS-SD on the local subnet. No servers, accounts, or internet connection needed.
- **Zero configuration.** Peers appear automatically. There is nothing to install per-project, no folder IDs to share, no repositories to initialise.
- **Automatic version backups.** Every overwrite saves the old file first. This is built into the synchronization protocol, not a feature that must be turned on.
- **Low-resource split architecture.** Go handles networking (peer coordination, TCP connections). C++ handles file I/O (watching, hashing, disk writes). This keeps CPU and memory use low on student laptops.

Three design principles guide the project:

1. **Transparency.** The synchronized folder should look and feel like a normal local folder. No special commands. No merge dialogs.
2. **Safety.** If the system changes a file, the old version must be recoverable. No automation should cause permanent data loss.
3. **Lightness.** The tool runs continuously in the background. It must not drain battery life or make the laptop feel slow.

2.5 Summary

The literature review taught us three things that directly determined the design of this project.

The CAP theorem says we must accept eventual consistency. Students disconnect constantly, and a synchronization tool that stops working when someone closes their laptop is useless. The system has to let everyone work offline and sort out differences later.

For conflict resolution, the right choice depends on scale and timeline. LWW with backups gives us simple, predictable behaviour with no risk of permanent data loss. CRDTs would be better for a production system, but they are complex to implement correctly. The schema supports adding them later.

No existing tool occupies the design space we are targeting. Cloud services, version control, P2P sync, and local sharing each solve part of the problem. None of them solve all of it at once, for a small team, on a local network, with no setup. That is where our project fits.

Chapter 3

State of the Art and Design Choices

3.1 Summary

This chapter documents the state of the art that emerged from our research. It covers the technology choices, architecture decisions, and synchronization strategies adopted for this project, with citations and critical assessment for each. Where Chapter 2 surveyed the landscape and identified the gap, this chapter explains how the research translated into concrete design decisions. Table 3.1 shows how each research finding maps to a component in the system.

Table 3.1: Research-to-Implementation Mapping

Research Finding	Source	System Component
Zero-config discovery	RFC 6762/6763	Go mDNS broadcaster
Causal event ordering	Lamport (1978)	Per-file Lamport counters
Partition tolerance	CAP theorem (2002)	Offline-first availability
Conflict resolution	Bayou (1995)	LWW + local backups
Optimistic replication	Saito & Shapiro (2005)	State-transfer architecture
Delta sync (future)	rsync (1996)	Planned block-level sync
File change detection	inotify / FSEvents	C++ kernel watchers
Data integrity	SHA-256 / TCP	C++ hashing + Go TCP pools

3.2 Zero-Configuration Peer Discovery

Multicast DNS (mDNS) and DNS-Based Service Discovery (DNS-SD) (Cheshire & Krochmal, 2013b, 2013a) are the protocols that allow devices on a local network to find each other without a central server. They are the reason a printer shows up on your laptop’s Wi-Fi list without

anyone typing in its IP address.

Our Go discovery component broadcasts a service record under the `_p2psync._tcp` domain and continuously browses the subnet for other instances. When a new peer appears, its address is resolved automatically. Users never see an IP address, a device ID, or an invitation code.

Why mDNS and not something else? During the research phase, Nizar looked into how Syncthing handles discovery. Syncthing uses a combination of local discovery and global relay servers. The local part is effective, but the global relay adds complexity and an external dependency. Since our target environment is a campus Wi-Fi where all team members are on the same subnet, mDNS gives us what we need without any external infrastructure (Syncthing Community, 2026).

There is a known limitation. Some enterprise Wi-Fi networks block multicast traffic. On EPITA's campus network, multicast works without issues. For networks that block it, a fallback would be needed, but adding a relay server would contradict the serverless design goal. For now, we accept this limitation and document it.

3.3 Event Ordering with Lamport Clocks

Every file in the system carries a Lamport counter in its metadata (Lamport, 1978). When a peer modifies a file, it increments the counter and includes the new value in the synchronization message. A receiving peer compares the incoming counter against its local value. If the incoming number is higher, the file is newer.

This is the same mechanism described in Section 2.2.1, applied at the per-file level. The counter is stored in the local SQLite metadata database alongside the file's SHA-256 hash and path.

One design question came up during implementation: should the counter be global (one counter per peer, shared across all files) or per-file? A global counter would increment even when unrelated files change, leading to unnecessary updates. A per-file counter only increments when that specific file changes, which keeps the update traffic proportional to actual changes. After discussing it in a team meeting, the per-file approach won.

The limitation is unchanged: Lamport clocks cannot detect concurrent edits. If Arihant edits `report.pdf` on his laptop while Huizhe edits the same file on his, and neither has seen the other's change, the timestamps do not tell the system that a conflict exists. The LWW tiebreaker picks one version, and the backup mechanism saves the other. Fidge's (1988) vector clocks would detect the concurrency, but at the cost of carrying an array of counters that must be updated whenever the peer group changes.

3.4 Availability Over Consistency

The CAP theorem (Gilbert & Lynch, 2002) forced a clear decision: prioritise availability. Peers must be able to edit files freely whether or not other peers are reachable. Reconciliation happens when connectivity returns.

In practice, this means the system is “eventually consistent” in the sense described by Vogels (2009). A peer that goes offline for an hour can accumulate many local changes. When it reconnects, the Go daemon exchanges metadata with other peers and pulls in any files that have newer timestamps. If a conflict exists, LWW resolves it automatically and the backup directory preserves the losing version.

The risk of eventual consistency is that two peers may briefly see different versions of the same file. For our use case, this is acceptable. A project team member seeing a file from ten seconds ago is fine. A system that prevents editing during a Wi-Fi dropout is not.

3.5 Last-Write-Wins with Version Backups

Conflict resolution uses LWW, following the approach from Bayou (Petersen et al., 1995). Before the C++ daemon writes an incoming file to disk, it copies the existing local version into a `.versions/` subdirectory, tagged with a timestamp. This ensures that LWW never causes permanent data loss.

Saito and Shapiro (2005) classify our approach as “state transfer with rollback support.” State transfer means sending entire file states (as opposed to operation transfer, which sends edit deltas). It is simpler to implement and debug, though it uses more bandwidth.

The team considered three alternatives before settling on LWW:

1. **Manual conflict resolution** (like Git): rejected because it requires user intervention, which defeats the goal of invisible synchronization.
2. **CRDTs** (Shapiro et al., 2011): rejected for the initial version due to implementation complexity, but the metadata schema is designed to support register-based CRDTs in a future iteration.
3. **Rename-on-conflict** (like Dropbox’s “conflicted copy”): considered, but it creates confusing duplicate files. LWW with backups achieves the same safety without cluttering the working directory.

3.6 File Change Detection

Polling a directory for changes is wasteful. It consumes CPU cycles even when nothing has changed, and the polling interval adds latency. Modern operating systems provide kernel-level event APIs: `inotify` on Linux (Kerrisk, 2024) and `FSEvents` on macOS (Apple, 2024).

The C++ daemon registers watchers on the synchronization directory and all its subdirectories. When a file is created, modified, or deleted, the kernel pushes an event. The daemon debounces rapid sequences of events (common when saving a file triggers multiple write operations) and forwards a consolidated update to the Go daemon through IPC.

This gives near-instant detection. In informal tests on Arihant’s MacBook, the delay between saving a file and the Go daemon receiving the metadata update was consistently under 50 milliseconds.

3.7 Data Integrity and Network Transport

Every file has a SHA-256 hash (National Institute of Standards and Technology, 2015) computed by the C++ daemon using memory-mapped I/O. The hash serves two purposes: detecting changes (if the hash is different, the file has changed) and verifying integrity after transfer (the receiver recomputes the hash and compares).

Network transport uses TCP (Eddy, 2022). The Go daemon maintains a pool of TCP connections with discovered peers. When a file transfer is needed, the Go daemon establishes the connection and hands the socket to the C++ daemon, which streams the file data directly to disk. TCP's built-in flow control and error recovery mean we do not need to implement our own retransmission logic.

Whole-file transfer is the initial design. The rsync algorithm (Tridgell & Mackerras, 1996) could reduce bandwidth by transferring only changed blocks, but implementing it correctly adds significant complexity. The architecture has a clean separation between the networking layer (Go) and the storage layer (C++), so adding block-level transfers later would only require changes in the C++ daemon.

Chapter 4

System Architecture

Summary

This chapter describes the design adopted to achieve the objectives stated in the Introduction. The architecture splits the system into two cooperating processes, a Go networking daemon and a C++ storage daemon, connected by inter-process communication. A Java frontend provides the user interface. Figure 4.1 shows how the components relate to each other.

4.1 System Architecture Anticipated

4.1.1 Component Overview

The decision to use two separate daemons was not arbitrary. Go has strong support for networking, concurrency, and cross-platform compilation, making it well-suited for peer discovery and synchronization coordination. C++ provides the low-level control needed for memory-mapped file I/O, kernel event APIs, and high-performance hashing. Running them as separate processes also means a crash in one does not take down the other.

1. **Go Networking Daemon.** Responsible for mDNS/DNS-SD peer discovery, managing TCP connection pools with discovered peers, coordinating which files need to be synchronised, and running the IPC server that the C++ daemon and Java frontend connect to.
2. **C++ Storage Daemon.** Responsible for kernel-level file watching (inotify on Linux, FSEvents on macOS), computing SHA-256 hashes with memory-mapped I/O, creating

version backups before overwrites, and streaming incoming files directly to disk from TCP sockets.

3. **Java Frontend.** A desktop application showing the current synchronization status, the list of connected peers, a file-by-file version history, and controls for choosing the synchronization directory. It communicates with the Go daemon through IPC.

4.1.2 Architecture Diagram

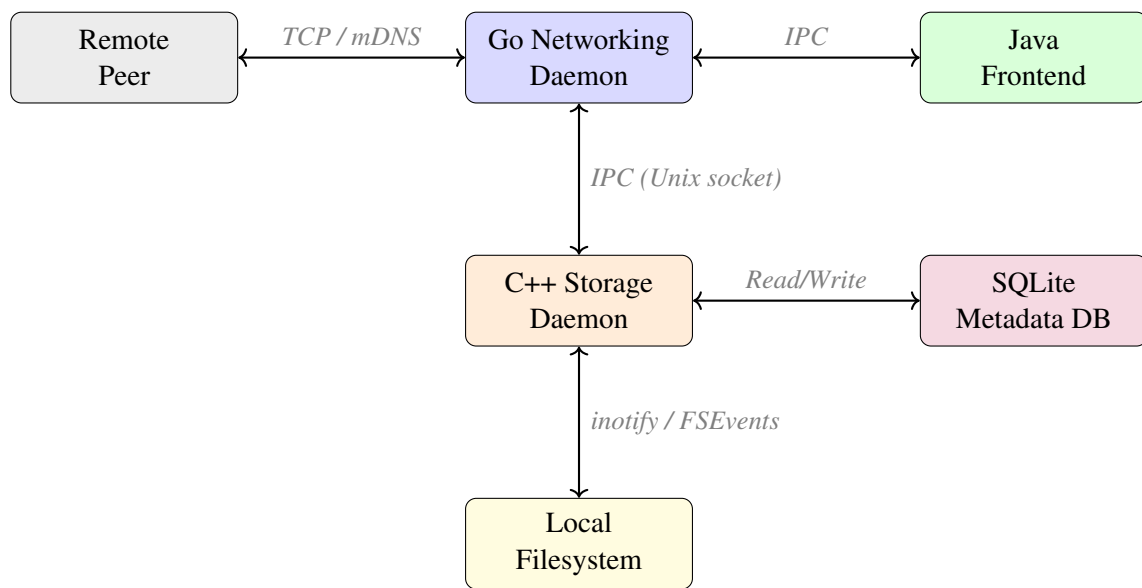


Figure 4.1: System architecture showing the Go networking daemon, C++ storage daemon, Java frontend, and their communication channels.

4.1.3 Synchronization Data Flow

A typical synchronization cycle proceeds through eight steps:

1. The C++ daemon detects a file change through a kernel event and computes the new SHA-256 hash.
2. It sends a metadata update (file path, new hash, incremented Lamport timestamp) to the Go daemon through IPC.
3. The Go daemon broadcasts the metadata to all connected peers over TCP.

4. Each receiving peer compares the incoming timestamp against its local copy.
5. If the incoming version is newer, the receiving Go daemon requests the full file from the sender.
6. The file data is streamed over TCP directly to the receiving C++ daemon.
7. Before writing the incoming file to disk, the receiving C++ daemon copies the existing version to the `.versions/` directory.
8. The new file is written, and the local SQLite metadata database is updated.

4.1.4 Metadata Storage

Each peer maintains a local SQLite database. The schema stores one row per tracked file:

- **path:** file path relative to the sync root
- **sha256:** content hash (32 bytes, hex-encoded)
- **lamport_ts:** the file's Lamport clock value (integer)
- **mtime:** last modification time (for display; not used for ordering)
- **size:** file size in bytes

The schema was designed with future extensibility in mind. Adding columns for CRDT state (e.g., a vector of per-peer counters) would not require restructuring the synchronization protocol.

4.1.5 Communication Protocols

Four communication channels are used:

- **Discovery:** mDNS multicast on port 5353, advertising the `_p2psync._tcp` service type.
- **Coordination:** A lightweight custom protocol over TCP for exchanging file metadata (file lists, version comparisons, transfer requests).

- **File Transfer:** Direct TCP streaming. The Go daemon sets up the connection; the C++ daemon handles the actual file I/O.
- **Internal IPC:** Unix domain sockets between the Go and C++ daemons on the same machine.

4.2 System Architecture Updated

This section will be completed at the end of Action Learning Week. It will document changes to the architecture that emerged during implementation: what worked as planned, what had to be modified, and what lessons came from the integration process.

Chapter 5

Methodology

Summary

This chapter describes the methodology used in the project, the stages by which it was implemented, and the research design. It details the participants, the tools and instruments used, the development procedure, and how results will be evaluated. The action plan for Action Learning Week is included at the end.

5.1 Methodology Anticipated

5.1.1 Team Structure and Roles

The team has four members. Responsibilities were divided by technical domain, based on each member's existing skills and what they wanted to learn:

- **Arihant Jain** (Backend, Go): peer discovery, synchronization coordination, IPC server, TCP networking. Arihant had prior experience with Go from a personal project and took the lead on the networking daemon.
- **Su Huizhe** (Backend, C++): file watching, SHA-256 hashing, file versioning, disk I/O. Huizhe had worked with C++ in systems programming courses and was comfortable with memory-mapped I/O and kernel APIs.
- **Nizar Soufiya** (Frontend, Java): user interface design and implementation. Nizar had experience with Java Swing from a previous course project.

- **Shengkang Duan** (Frontend and CI/CD, Java): build automation, test pipeline, GitHub Actions configuration. Shengkang set up the continuous integration pipeline early in the project, which gave the team fast feedback on every commit.

The split between backend and frontend roughly corresponds to the Go/C++ daemons and the Java GUI described in Chapter 4. The two groups worked in parallel for most of the project, meeting weekly to align on interface contracts and integration issues.

5.1.2 Development Process

Rather than writing a detailed specification and implementing it sequentially, the team followed an iterative development process structured around the course milestones from April to July 2026. The project moved through four distinct phases:

1. **Phase 1: Initiation and Research (April 19 to May 15, 2026).** Began with the Project Proposal submission on April 19, followed by the Literature Review and Presentation Workshop on April 28. During this phase, we divided research areas, submitted our initial bibliography on May 8, and outlined the literature review on May 15. The focus was on studying distributed systems algorithms (such as logical clocks and CAP theorem trade-offs) and testing local network discovery protocols (mDNS/DNS-SD).
2. **Phase 2: Core Prototyping (May 6 to May 29, 2026).** Initiated at Hackathon 1 (Prototype Sprint) on May 6, where we wrote the first standalone prototypes of the Go networking module and the C++ file-watching daemon. The theoretical foundation was finalized and submitted as the Completed Literature Review on May 29.
3. **Phase 3: Integration and MVP Sprints (June 12 to June 22, 2026).** Following the Public Speaking Workshop on June 12, we completed Hackathon 2 (MVP Sprint) on June 17, which focused on end-to-end integration over Unix domain sockets. This phase culminated in the submission of this thesis and the accompanying Gen AI documentation on June 22.

4. **Phase 4: Live Evaluation (July 6 to July 10, 2026).** Scheduled for Action Learning Week, this final phase consists of live WLAN latency and consistency testing, concluding on July 10 with the submission of the final MVP, 360 peer evaluations, and poster presentation.

Table 5.1: Action Learning Project Milestones

Milestone / Event	Date / Deadline
Project Proposal Submission	April 19, 2026
Literature Review Workshop	April 28, 2026 (In-person)
Hackathon 1 (Prototype Sprint)	May 6, 2026 (In-person)
Bibliography Submission	May 8, 2026
Literature Review Outline	May 15, 2026
Completed Literature Review	May 29, 2026
Public Speaking Workshop	June 12, 2026
Hackathon 2 (MVP Sprint)	June 17, 2026 (In-person)
Thesis & Gen AI Submission	June 22, 2026
Action Learning Week	July 6 to 10, 2026 (In-person)
Final MVP, Poster & 360 Eval	July 10, 2026

5.1.3 Tools and Instruments

5.1.4 Research Design

The project tests two hypotheses through controlled experiments:

Hypothesis 1 (Latency): Measure the time from a file being saved on one peer to it appearing on another, across 3 to 5 peers connected to the same WLAN. Success is defined as

Table 5.2: Development Tools

Category	Tool
Languages	Go 1.21+, C++17, Java 17
Build systems	Go modules, CMake (C++), Gradle (Java)
Version control	Git with GitHub for hosting and code review
CI/CD	GitHub Actions (build, lint, test on every push)
Testing frameworks	Go <code>testing</code> package, Google Test (C++), JUnit 5 (Java)
Test environment	3 to 5 laptops on the same WLAN subnet
Documentation	L ^A T _E X (thesis and literature review)

sub-second average latency for files up to 10 MB. The test procedure involves saving files of different sizes (1 KB, 100 KB, 1 MB, 10 MB) and recording timestamps at both ends.

Hypothesis 2 (Consistency): Simulate concurrent edits by having two peers modify the same file while disconnected, then reconnecting them. Success is defined by three criteria: (a) no file is permanently lost due to conflict resolution, (b) all peers converge to the same version within 30 seconds of reconnection, and (c) the overwritten version is recoverable from the `.versions/` backup directory.

5.1.5 Action Plan

Table 5.3 maps the work to the Action Learning Week timeline.

Table 5.3: Action Plan for Action Learning Week

Period	Planned Work
Pre-ALW	Literature review complete, architecture finalised, Go and C++ prototypes built and tested independently
ALW Days 1 and 2	IPC integration between Go and C++ daemons; first end-to-end sync test between two laptops
ALW Days 3 and 4	Multi-peer testing (3+ laptops); Java frontend integration; conflict simulation tests
ALW Day 5	Hypothesis evaluation (latency testing, consistency validation); documentation of results

5.2 Methodology Updated

This section will be completed at the end of Action Learning Week. It will describe what actually happened: which parts of the plan worked, what had to change, what problems came up, and how the team adapted.

Chapter 6

Results

Summary

This chapter details the results of what was learned and accomplished during the project.

6.1 Development Anticipated

Based on the architecture in Chapter 4 and the methodology in Chapter 5, the following development outcomes are planned:

Go Networking Daemon. A working daemon that discovers peers on the local subnet through mDNS/DNS-SD, maintains persistent TCP connections with them, exchanges file metadata to detect changes, coordinates file transfers, and communicates with the C++ daemon through Unix domain sockets. The core packages (`discovery`, `network`, `ipc`) are already prototyped and tested independently; integration is the remaining work.

C++ Storage Daemon. A working daemon that watches the synchronization directory using kernel-level APIs (`inotify` on Linux, `FSEvents` on macOS), computes SHA-256 hashes of modified files with memory-mapped I/O, saves existing files to a `.versions/` directory before any overwrite, and streams incoming file data directly from TCP sockets to disk.

Java Frontend. A desktop application that shows the current synchronization status (which peers are connected, which files are syncing), displays a per-file version history with the ability to restore a previous version, and lets users configure the synchronization directory.

Hypothesis Testing. Two sets of measurements:

- H_1 (Latency): timed synchronization tests across 3 to 5 peers on the campus WLAN, for files ranging from 1 KB to 10 MB. Target: sub-second average latency.

- H_2 (Consistency): concurrent edit scenarios where two peers modify the same file while disconnected, then reconnect. Expected outcome: automatic resolution with no permanent data loss, convergence within 30 seconds, and full recoverability from backups.

6.2 Development Updated

This section will be completed at the end of Action Learning Week in a separate standalone document. It will describe the differences between what was anticipated above and what was actually developed: features that were completed, features that were modified, and features that were deferred. Maximum one-half page.

Chapter 7

Conclusions

When this project started, the problem seemed straightforward: build a tool that lets a small group of students share files on a local network without setting up servers or learning Git. The research and design process revealed that the problem is more nuanced than it appears. Distributed synchronization, even for a handful of peers, touches on event ordering, consistency models, and conflict resolution, all topics with decades of research behind them.

The literature review (Chapter 2) showed that existing tools each solve part of the problem. Cloud services handle synchronization well but depend on external infrastructure. Git handles versioning well but requires technical expertise. Synthing handles peer-to-peer transfer well but treats versioning as an afterthought. No existing tool scores well across all five evaluation dimensions (membership, synchronization, history, file support, and transparency) simultaneously.

The system described in this thesis fills that gap. The Go networking daemon discovers peers through mDNS/DNS-SD and coordinates synchronization using per-file Lamport clocks with Last-Write-Wins conflict resolution. The C++ storage daemon watches the filesystem for changes, computes content hashes, and creates backups before every overwrite. The Java frontend shows users what is happening without requiring them to understand the underlying protocols.

The design makes three deliberate trade-offs:

- **Lamport clocks instead of vector clocks.** Simpler to implement and manage when peers join and leave, but unable to detect truly concurrent edits.
- **LWW instead of CRDTs.** One version is chosen automatically; the other is preserved in a backup. Simple and predictable, but not conflict-free in the CRDT sense.

- **Whole-file transfers instead of delta sync.** Easy to implement and debug, but wasteful for large files with small changes.

Each of these trade-offs is appropriate for 3 to 5 peers on a local network with a project timeline of a few months. A production system serving hundreds of users would need to revisit all three.

Several directions for future work remain:

- **Block-level delta synchronization.** Implementing the rsync rolling-checksum algorithm in the C++ daemon to transfer only changed blocks, reducing bandwidth for large files.
- **CRDT-based metadata.** Extending the SQLite schema to use state-based CRDTs for file metadata, enabling conflict-free convergence without LWW.
- **Cross-subnet discovery.** Supporting peers on different subnets, possibly through a lightweight relay, while preserving zero-configuration on local networks.
- **Mobile support.** Adapting the architecture for Android and iOS, where background execution and filesystem access are more restricted than on desktop operating systems.

What this project demonstrates is that the individual building blocks for lightweight peer-to-peer synchronization, mDNS, Lamport clocks, LWW, TCP, SHA-256, are well-understood and well-documented. The challenge is not inventing new algorithms. It is combining existing ones into a package that is simple enough for non-technical users, safe enough that no data is lost, and light enough to run unnoticed on a student laptop. That is what this system sets out to do.

References

- Apple. (2024). *FSEvents reference*. https://developer.apple.com/documentation/coreservices/file_system_events. (Apple Developer Documentation. Accessed: 2026-06-20)
- Chacon, S., & Straub, B. (2014). *Pro git* (2nd ed.). Apress.
- Cheshire, S., & Krochmal, M. (2013a). *DNS-based service discovery*. RFC 6763. Retrieved from <https://www.rfc-editor.org/rfc/rfc6763> (Accessed: 2026-05-28)
- Cheshire, S., & Krochmal, M. (2013b). *Multicast DNS*. RFC 6762. Retrieved from <https://www.rfc-editor.org/rfc/rfc6762> (Accessed: 2026-05-28)
- Demers, A., Greene, D., Hauser, C., Irish, W., Larson, J., Shenker, S., ... Terry, D. (1987). Epidemic algorithms for replicated database maintenance. In *Proceedings of the sixth annual acm symposium on principles of distributed computing* (pp. 1–12). doi: 10.1145/41840.41841
- Dropbox. (2026). *Dropbox*. <https://www.dropbox.com/>. (Cloud-based file hosting and synchronization platform. Accessed: 2026-06-20)
- Eddy, W. M. (2022). *Transmission control protocol (TCP)*. RFC 9293. Retrieved from <https://www.rfc-editor.org/rfc/rfc9293> (Accessed: 2026-05-28)
- Fidge, C. J. (1988). Timestamps in message-passing systems that preserve the partial ordering. In *Proceedings of the 11th australian computer science conference* (pp. 56–66).
- Gilbert, S., & Lynch, N. (2002). Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2), 51–59.
- Google. (2026). *Google drive*. <https://www.google.com/drive/>. (Cloud storage and file synchronization service. Accessed: 2026-06-20)
- Jayakar, S. (2020). *Rewriting the heart of our sync engine*. <https://dropbox.tech/infrastructure/rewriting-the-heart-of-our-sync-engine>. (Dropbox Engineering Blog. Accessed: 2026-06-20)
- Kerrisk, M. (2024). *inotify(7) — linux manual page*. <https://man7.org/linux/man-pages/>

- man7/inotify.7.html. (Accessed: 2026-05-28)
- Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7), 558–565.
- National Institute of Standards and Technology. (2015). *FIPS PUB 180-4: Secure hash standard (SHS)*. Retrieved from <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf> (Accessed: 2026-05-28)
- Parker, D. S., Popek, G. J., Rudisin, G., Stoughton, A., Walker, B. J., Walton, E., . . . Kline, C. (1983). Detection of mutual inconsistency in distributed systems. *IEEE Transactions on Software Engineering*, SE-9(3), 240–247. doi: 10.1109/TSE.1983.236733
- Petersen, K., Spreitzer, M., Terry, D., Theimer, M., & Demers, A. (1995). Managing update conflicts in Bayou, a weakly connected replicated storage system. In *Proceedings of the 15th acm symposium on operating systems principles* (pp. 172–182). doi: 10.1145/224056.224070
- Resilio, Inc. (2026). *Resilio sync*. <https://www.resilio.com/sync/>. (Peer-to-peer file synchronization software using modified BitTorrent protocol. Accessed: 2026-06-20)
- Saito, Y., & Shapiro, M. (2005). Optimistic replication. *ACM Computing Surveys*, 37(1), 42–81. doi: 10.1145/1057977.1057980
- Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). A comprehensive study of convergent and commutative replicated data types. *INRIA Research Report*(RR-7506).
- Sun, C., & Ellis, C. (1998). Operational transformation in real-time group editors: Issues, algorithms, and achievements. In *Proceedings of the acm conference on computer supported cooperative work* (pp. 59–68). doi: 10.1145/289444.289469
- Syncthing Community. (2026). *Syncthing documentation*. <https://docs.syncthing.net/>. (Open-source continuous file synchronization program. Accessed: 2026-06-20)
- Terry, D. B., Demers, A. J., Petersen, K., Spreitzer, M. J., Theimer, M. M., & Welch, B. B. (1994). Session guarantees for weakly consistent replicated data. In *Proceedings of the third international conference on parallel and distributed information systems* (pp. 140–149). doi: 10.1109/PDIS.1994.331722
- Tridgell, A., & Mackerras, P. (1996). The rsync algorithm. In *Proceedings of the unix annual*

technical conference.

Vogels, W. (2009). Eventually consistent. *Communications of the ACM*, 52(1), 40–44. doi:
10.1145/1435417.1435432

Appendices

Appendix A

Detailed Comparison Tables

The following tables provide a detailed, dimension-by-dimension comparison of existing collaborative file management approaches. They supplement the analysis presented in Chapter 2 by providing granular ratings for individual systems within each category.

The qualitative ratings used are defined in Table A.1.

Table A.1: Qualitative Rating Scale

Rating	Meaning
High	Strong support for the capability
Partial	Partial support with notable limitations
Limited	Basic or restricted support
None	Capability not provided

Table A.2: Comparison: Membership Management

Category	System	Peer Discovery	Access Control	Group Formation
Centralized	<i>Google Drive</i>	Limited	High	High
	<i>OneDrive</i>	Limited	High	High
	<i>Dropbox</i>	Limited	High	High
	<i>Nextcloud</i>	Limited	High	Partial
Version Control	<i>Git</i>	None	None	Limited
	<i>SVN</i>	None	High	Limited
	<i>Perforce</i>	None	High	Limited
P2P Sync	<i>Syncthing</i>	High	Partial	Partial
	<i>Resilio Sync</i>	Partial	Partial	Partial
Local Sharing	<i>AirDrop</i>	High	None	Limited
	<i>Nearby</i>	High	None	Limited
	<i>Share</i>			

Table A.3: Comparison: Synchronization Model

Category	System	Automatic Sync	Offline Operation	Multi-Device Sync
Centralized	<i>Google Drive</i>	High	Limited	High
	<i>OneDrive</i>	High	Limited	High
	<i>Dropbox</i>	High	Limited	High
	<i>Nextcloud</i>	High	Partial	High
Version Control	<i>Git</i>	None	High	Partial
	<i>SVN</i>	None	Limited	Partial
	<i>Perforce</i>	None	Limited	Partial
P2P Sync	<i>Syncthing</i>	High	High	High
	<i>Resilio Sync</i>	High	High	High
Local Sharing	<i>AirDrop</i>	None	None	None
	<i>Nearby Share</i>	None	None	None

Table A.4: Comparison: History Management

Category	System	Version History	Rollback	Conflict Resolution
Centralized	<i>Google Drive</i>	Limited	Limited	Limited
	<i>OneDrive</i>	Limited	Limited	Limited
	<i>Dropbox</i>	Limited	Limited	Limited
	<i>Nextcloud</i>	Partial	Partial	Limited
Version Control	<i>Git</i>	High	High	High
	<i>SVN</i>	High	High	High
	<i>Perforce</i>	High	High	High
P2P Sync	<i>Syncthing</i>	Partial	Partial	Limited
	<i>Resilio Sync</i>	Partial	Partial	Limited
Local Sharing	<i>AirDrop</i>	None	None	None
	<i>Nearby Share</i>	None	None	None

Table A.5: Comparison: File Support

Category	System	Heterogeneous Files	Large Files
Centralized	<i>Google Drive</i>	High	Partial
	<i>OneDrive</i>	High	Partial
	<i>Dropbox</i>	High	Partial
	<i>Nextcloud</i>	High	High
Version Control	<i>Git</i>	Partial	Limited
	<i>SVN</i>	Partial	Partial
	<i>Perforce</i>	High	High
P2P Sync	<i>Syncthing</i>	High	High
	<i>Resilio Sync</i>	High	High
Local Sharing	<i>AirDrop</i>	High	High
	<i>Nearby</i>	High	High
	<i>Share</i>		

Table A.6: Comparison: Filesystem Transparency and Usability

Category	System	Filesystem Integration	Ease of Use
Centralized	<i>Google Drive</i>	High	High
	<i>OneDrive</i>	High	High
	<i>Dropbox</i>	High	High
	<i>Nextcloud</i>	High	Partial
Version Control	<i>Git</i>	Limited	Limited
	<i>SVN</i>	Limited	Limited
	<i>Perforce</i>	Limited	Limited
P2P Sync	<i>Syncthing</i>	High	High
	<i>Resilio Sync</i>	High	High
Local Sharing	<i>AirDrop</i>	High	High
	<i>Nearby</i>	High	High
	<i>Share</i>		

Table A.7: Gap Summary Across Evaluation Dimensions

Approach	Membership Management	Sync	History Management	File Support	Filesystem Transparency
Centralized Platforms	High	High	Limited	High	High
Version Control Systems	Partial	Limited	High	Partial	Limited
P2P Sync Systems	Partial	High	Limited	High	High
Local Sharing	Limited	None	None	High	High
Desired Solution	High	High	High	High	High

Appendix B

Flesch Readability Statistics

As required by the Action Learning programme guidelines, this appendix presents the Flesch Readability Statistics for the thesis. The metrics were calculated across all seven chapters of the main text, excluding the bibliography, appendices, and LaTeX command markup.

The Flesch Reading Ease score of **38.72** indicates that the document is written at an appropriate academic level while maintaining readability. The Flesch-Kincaid Grade Level is **11.6**, corresponding to a high school senior level of comprehension, which reflects clear and concise technical communication.

Table B.1 lists the metrics for each chapter and the overall thesis.

Table B.1: Flesch Readability Statistics by Chapter

Chapter	Sentences	Words	Flesch Reading Ease	Flesch-Kincaid Grade
1. Introduction	40	676	34.62	12.6
2. Literature Review	142	1,882	43.51	10.5
3. State of the Art	77	1,178	39.34	11.6
4. System Architecture	50	684	42.18	10.8
5. Methodology	39	840	30.98	14.3
6. Results	17	293	21.58	14.5
7. Conclusions	33	463	38.01	11.4
Total Thesis Content	398	6,016	38.72	11.6

List of Figures

4.1	System architecture showing the Go networking daemon, C++ storage daemon, Java frontend, and their communication channels.	23
-----	---	----

List of Tables

3.1	Research-to-Implementation Mapping	17
5.1	Action Learning Project Milestones	28
5.2	Development Tools	29
5.3	Action Plan for Action Learning Week	29
A.1	Qualitative Rating Scale	39
A.2	Comparison: Membership Management	39
A.3	Comparison: Synchronization Model	40
A.4	Comparison: History Management	40
A.5	Comparison: File Support	41
A.6	Comparison: Filesystem Transparency and Usability	41
A.7	Gap Summary Across Evaluation Dimensions	41
B.1	Flesch Readability Statistics by Chapter	42